VLSI Verification Lab

Mohammad Attari

1 Introduction

In this lab you are going to get yourself acquainted with modern techniques when it comes to dealing with verification for the systems that you, or other people, have designed. You will be using SystemVerilog, which is an IEEE-standard language used in the idustry both for hardware description and verification (https://en.wikipedia.org/wiki/SystemVerilog).

In the following sections try to get yourself up to speed with the different material used in this lab. SystemVerilog is a huge language, so it might be daunting at first to confront the beast, but soldier on, as your VHDL background should be more than enough to aid you in your endeavors.

2 File tree

In this lab you are provided with a number of SystemVerilog files. These files can be visually structured according to the tree in Figure 1.

```
Design
└── serial_to_parallel.sv
Traditional testbench
└── test_top.sv
UVM testbench
├── clock_uvc.svh
├── reset_uvc.svh
├── scoreboard.svh
├── serial_data_uvc.svh
├── tb_env.svh
├── test.svh
├── top_config.svh
├── clock_if.sv
├── reset_if.sv
├── serial_data_if.sv
├── tb_pkg.sv
└── tb_top.sv
```

Figure 1: File tree

3 Design

The design unit in this lab is a very simple serial to parallel data converter, with an equally simple interface. Figure 2 depicts a timing diagram for two packets of data that are converted from serial to parallel format.

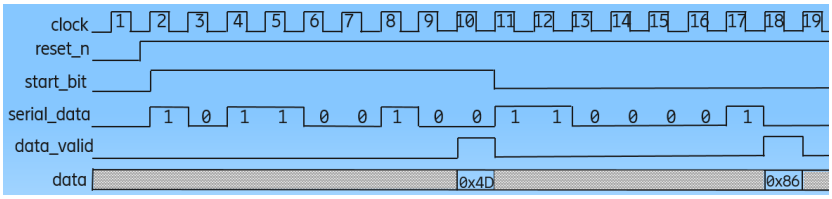


Figure 2: Timing diagram

The system leaves the reset state when the active low `reset_n` signal is deasserted (goes high). A new transaction is then triggered by the level-sensitive `start_bit`. As you can see, the least significant data bit comes first in time, followed by the more significant bits. So for instance, for the 2nd packet (that is 0x86) the first bit that arrives is 0 followed by 1, 1, 0, 0, 0, 0, and finally a 1. You can verify for yourself that 0x86 = 10000110. After one data bundle is converted, the output `data_valid` signal is asserted for one clock cycle.

This design is provided as just one SystemVerilog file (`serial_to_parallel.sv`), and the implementation is starightforward, so you should not have much difficulty in deciphering its contents. Make sure you have inspected the file and understand how it works before moving forward. Discuss among yourselves, and, if necessary, ask for help from the TAs.

4 Traditional testbench

The traditional testbench is also bundled in just one SystemVerilog file (`test_top.sv`). Here is a brief description of the different sections in the file, labeled according to the comments starting with the `// Section` in the file.

- Section 1: contains the internal signals and variables used by the testbench
- Section 2: initializes the internal signals and variables
- Section 3: generates a clock signal with a period of 10ns
- Section 4: maintains a state machine to keep track of the activity state of the system
- Section 5: contains a task that is used to generate the reset stimuli. The reset signal gets activated for a certain length after a certain amount of delay has passed (both in clock cycles)
- Section 6: similar to the previous section but this one generates the start signal stimuli
- Section 7: contains a task that generates the serial data stimuli. It takes an 8-bit value and after a certain amount of delay (in clock cycles) starts to deliver the bits, one at a time, to the serial data signal (least significant bit first)
- Section 8: contains a task that monitors and checks the correct functionality of the design under test. It also keeps a record of the number checked data and the number of errors (if any) that have occurred.
- Section 9: contains the functional coverage definitions. You will be tasked with changing its contents later on in the manual
- Section 10: instantiates the design under test
- Section 11: generates the stimuli by calling the appropriate stimulus generator task. You will modify this task later on in order to produce the necessary stimuli. But fear not! The `fork-join` statements start parallel tasks in SystemVerilog, so assume each `begin-end` block is running in parallel with the other blocks in the task (think of them as parallel threads)
- Section 12: starts the stimulus generator and checker tasks and additionally prints out the overall status of the tests when the other tasks are finished

5 UVM testbench

The Universal Verification Methodology (UVM) is a standard used in the industry as a verification methodology. The UVM testbench here is broken down into multiple files according to section 2.

6 Tasks

6.1 Create a Questa Sim project

In this task you will create a new project in Siemens' Questa Sim simulator. In order to do this, first navigate to this link in your browser: AppsAnywhere. You might be prompted to log in and open the launcher, as shown in Figure 3.

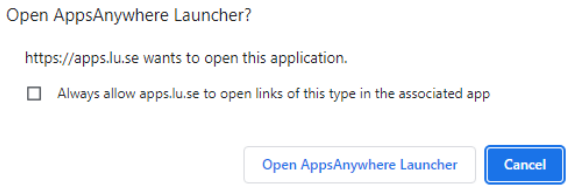


Figure 3: AppsAnywhere - Launcher

Click on `Open AppsAnywhere Launcher` button and you should be able to see a page similar to the one in Figure 4.

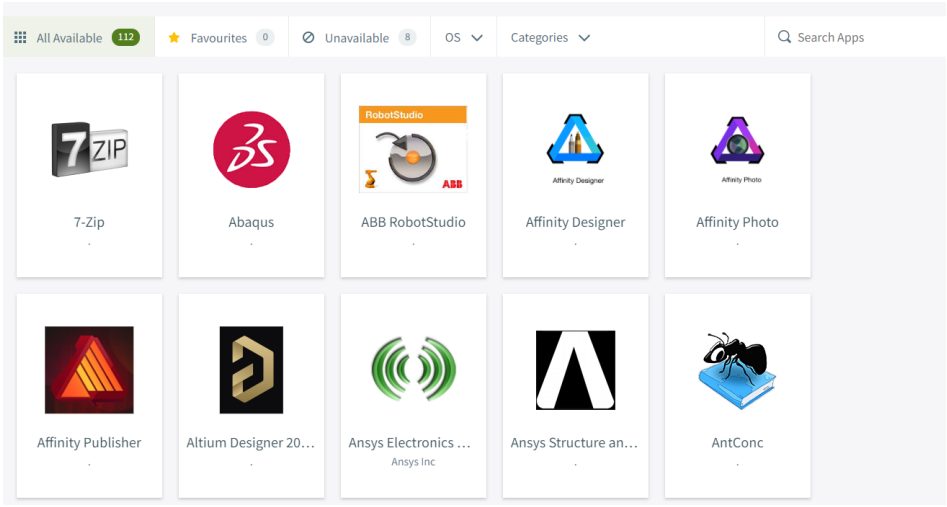


Figure 4: AppsAnywhere - Web Page

Now search for the Questa simulator by typing in the `Search Apps` text box in the upper right corner of the page (see Figure 5), and then hover over the `Questa Sim` frame and click on the `Launch` button (Figure 6).

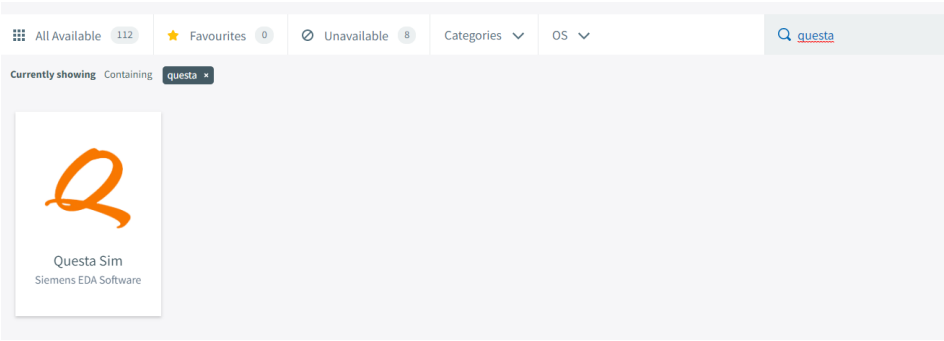


Figure 5: AppsAnywhere - Questa Sim

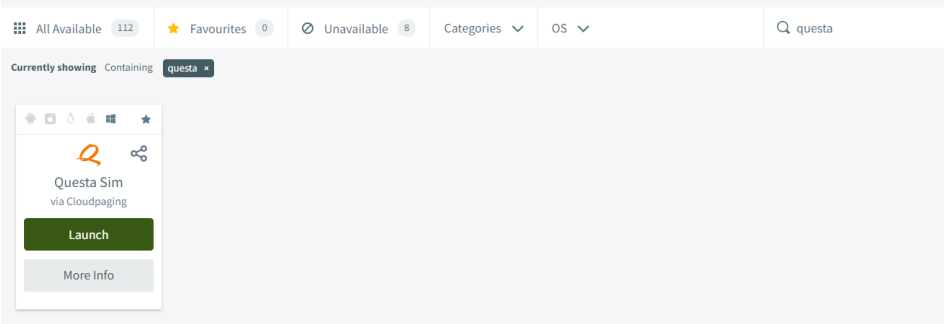


Figure 6: AppsAnywhere - Questa Sim Launch

You might be presented with another prompt similar to the one in Figure 3, and after accepting it you should be able to see the main window of the Questa Sim simulator, as depicted in Figure 7.

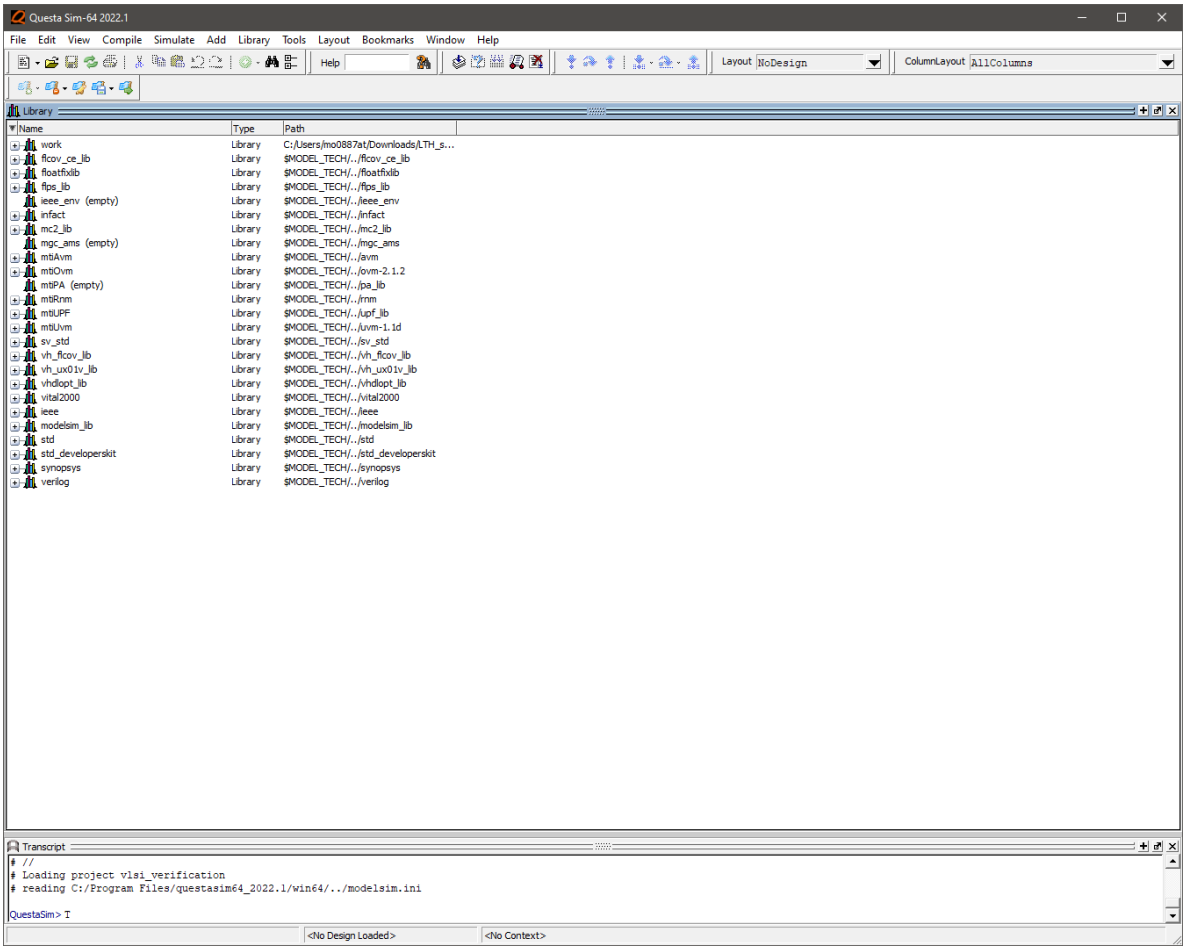


Figure 7: Questa Sim Window

Now try to create a new project by going through `File->New->Project...`, which will open up the `Create Project` window (Figure 8).

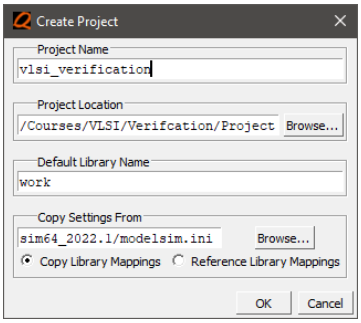


Figure 8: Create Project Window

Type in a proper project name and choose a suitable project location and then click `OK`. This will be followed by another window titled `Add items to the Project` (Figure 9).

Note: Do not create the project in `C:\` or in your student drive. Use a local folder for the project location instead.

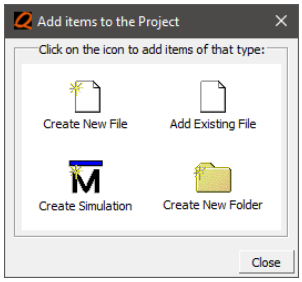


Figure 9: Add items window

Click on `Add Existing File`, which will open the `Add file to Project` window (Figure 10), and try to add all the `.sv` files to the project (skip the `.svh` header files). Keep in mind that by default the `Reference from current location` radio button (at the bottom) is selected, which means that the files that you add will not be copied to the project directory. If you want to replicate the files in the project directory, then make sure that the `Copy to project directory` radio button is selected before you add the files. Although in this case you need to manually copy the `.svh` header files to the project directory yourself.

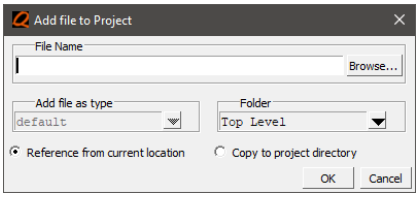


Figure 10: Add file window

Repeat this process untill all the `.sv` files are added to the project. You can add more files to the project later on if you desire by right clicking on an empty spot in the project window and then going through `Add to Project->Existing File...` (Figure 11), which will open the `Add file to Project` window (Figure 10) again.

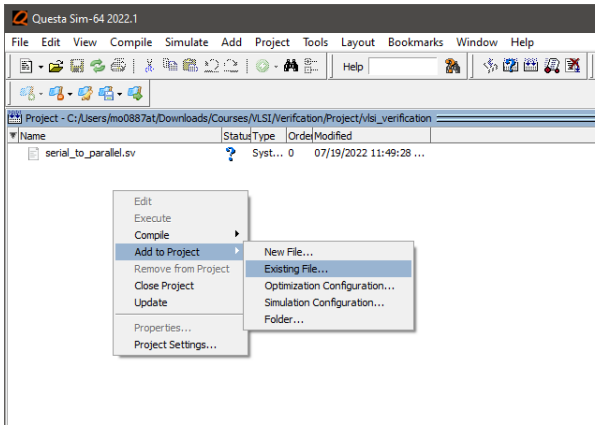


Figure 11: Add existing file

Now, compile all the files, either through the file menu by following `Compile->Compile All`, or by clicking directly on the `Compile All` button in the toolbar (Figure 12).



Figure 12: Compile All button

You should now be able to see a green tick mark next to each file in the project window, indicating successful compilation, similar to Figure 13.

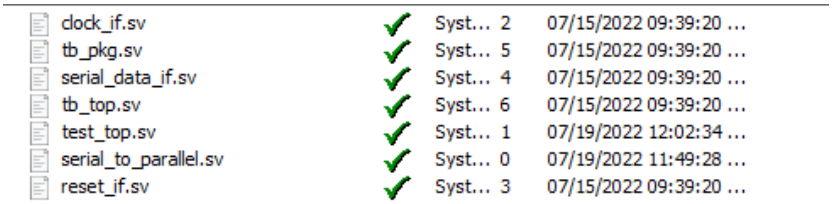


Figure 13: Successful compilation

Now that you have everything locked and loaded you can start the simulation for the traditional testbench. For this, navigate to the `Library` tab near the bottom left corner of the project window, and then expand the `work` library tree, right click on the `serial_to_parallel_test` module, and select `Simulate`, as illustrated in Figure 14.

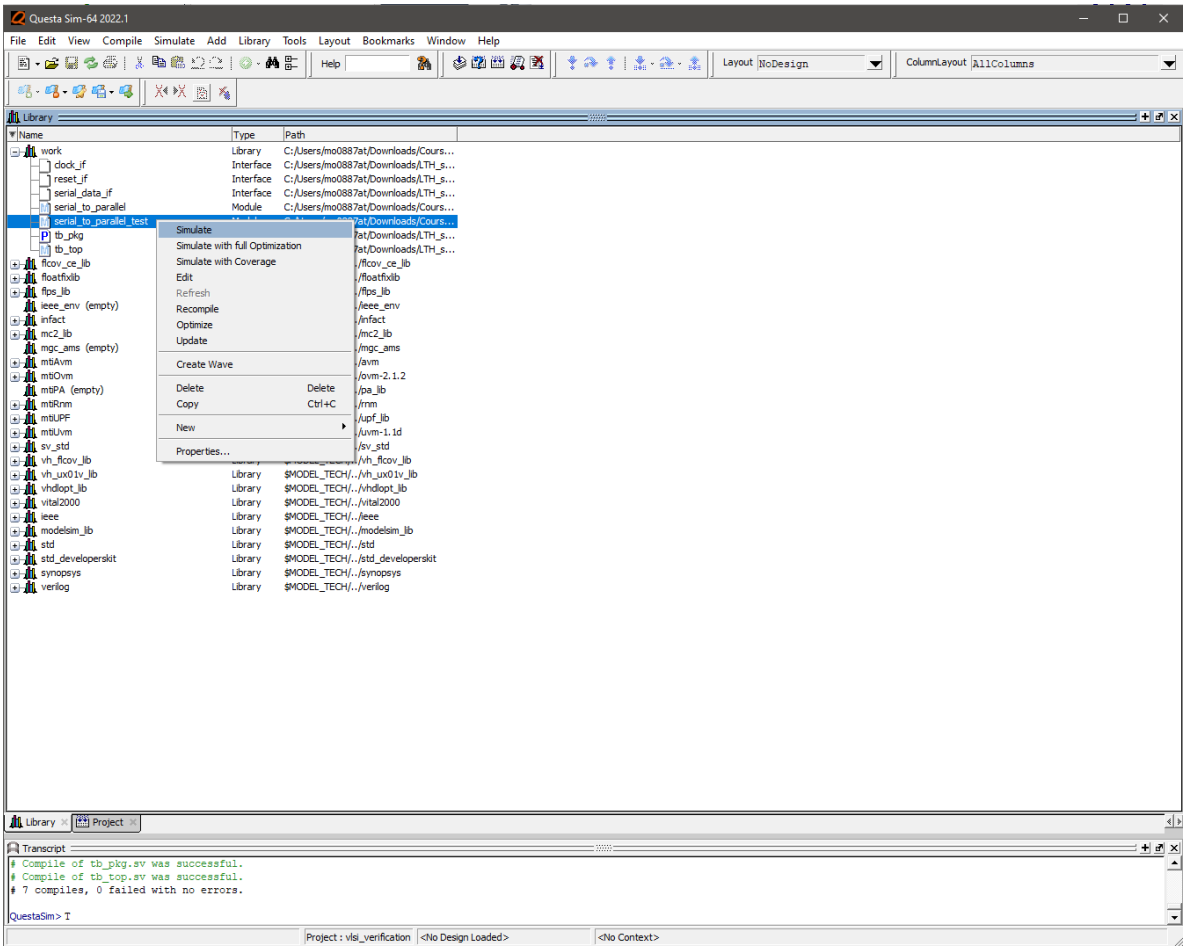


Figure 14: Start the simulation

This will cause the project window to metamorphose into Figure 15.

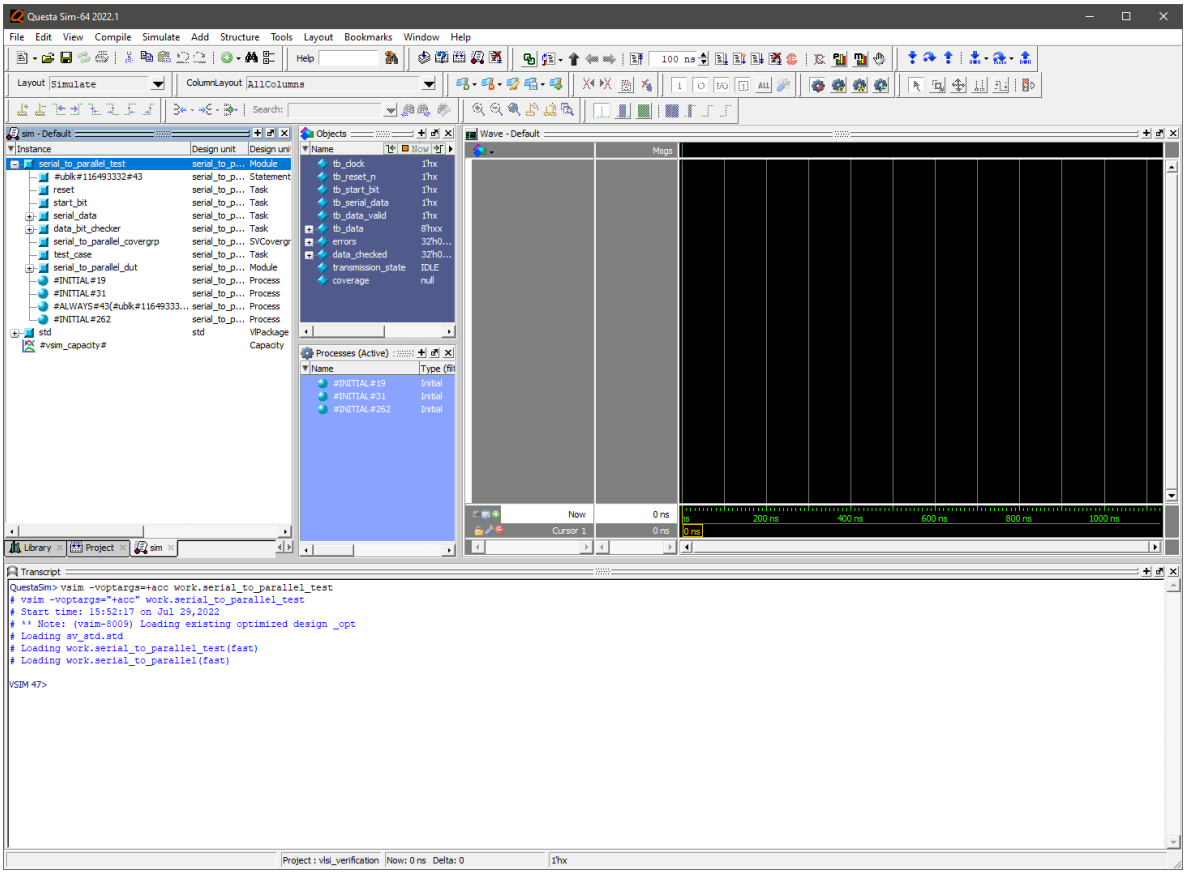


Figure 15: Simulation window

Click on the design under test module (`serial_to_parallel_dut`) in the `sim - Default` pane on the left, and then click on the `clock` signal in the `Objects` pane in the middle, followed by pressing `Ctrl+A` on the keyboard to select all the signals in the `Objects` pane, which highlights all the signals in the `Objects` pane with a white tint. Now right click on any of the signals in the `Objects` pane and click on `Add Wave` to add the selected signals to the `Wave - Default` pane on the right. Figure 16 shows what the project window should look like if you did not err in following the instructions above.

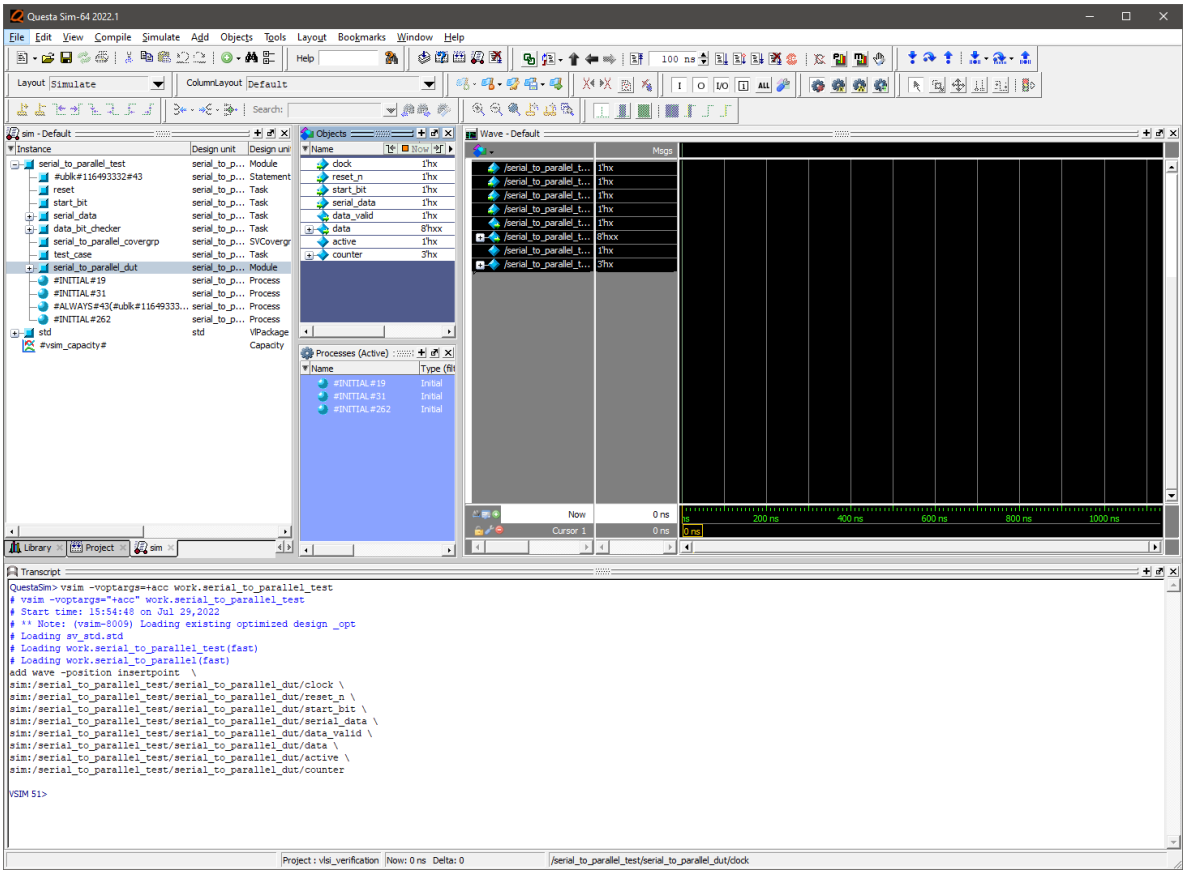


Figure 16: Simulation window - signals added to wave

Now you are going to simulate the design. In order to do this, you can either click through the file menu by following `Simulate->Run->Run -All`, or alternatively you can use the toolbar button in Figure 17. This will start the simulation and will stop only when all the tests have finished.

Note: If, by accident, you press this button again after simulation has finished, the simulation will run forever, and in that case you need to stop the simulation by pressing the `Stop` button (Figure 18).



Figure 17: Run -All button



Figure 18: Stop button

When the simulation is done you should have a window similar to Figure 19. If you inspect the transcript pane (at the bottom) you will notice the printouts informing you about how the test progressed. For now it says that the "TEST CASE PASSED" and that "FUNCTIONAL COVERAGE FAILED". Although, you should not read too much into this report yet, as the testbench is left in

an incomplete state (notice the message "Number of checked data 0") that you will remedy in the following tasks.

You can switch to the `Wave` pane (as was done in Figure 19) to inspect the waveform of the signals, which can be quite handy in debugging the design under test. Try to play around in the `Wave` pane a bit and make yourself familiar with how it works. For now the waveform is a bit bare, but you can still zoom in and out, jump to signal transitions, change the radix format of signals, and even do things like forcing a value onto a signal. If you have difficulties seek help from the TAs.

You can stop the simulation at any time by `Simulate->End Simulation`, but do not do it yet, as it is more convenient to keep it running, do the modifications, restart and run again, as you will see later on.

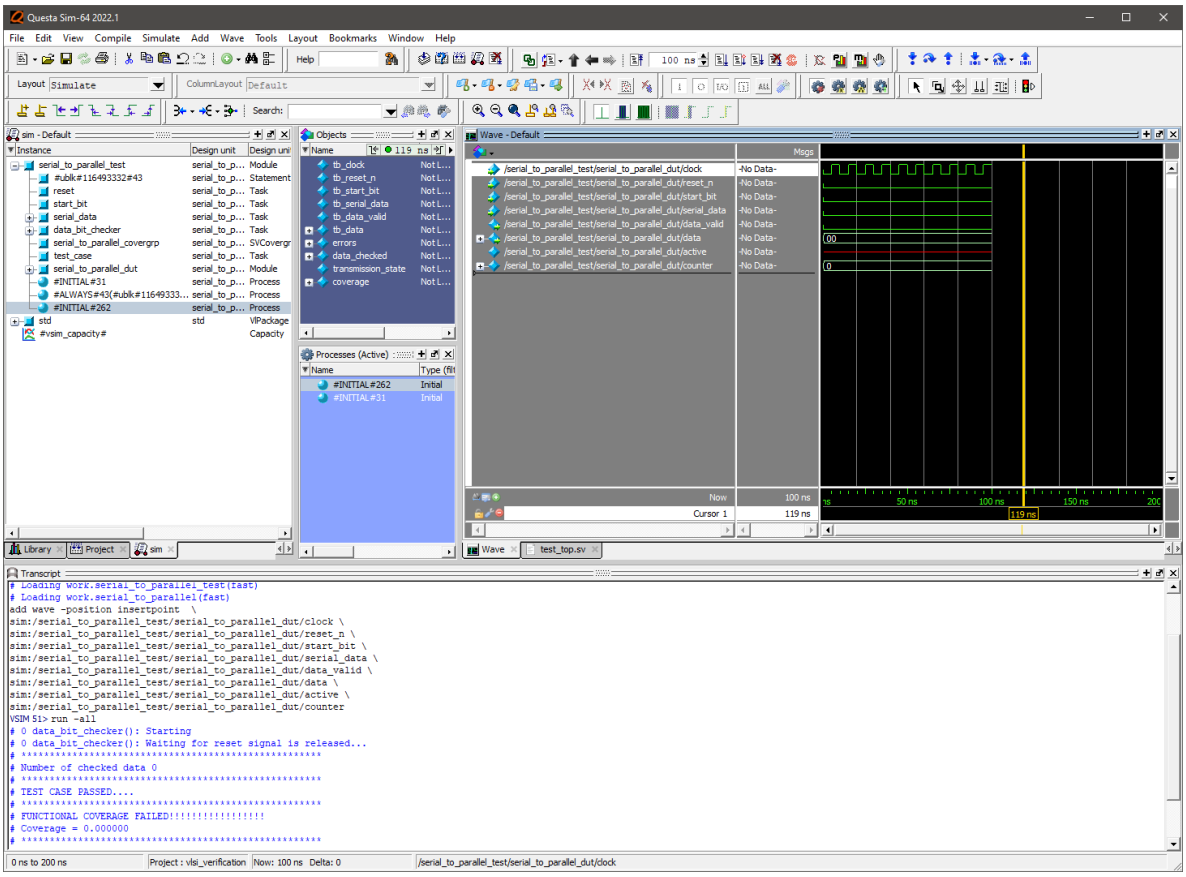


Figure 19: Finished simulation

6.2 Coverage for the reset signal

The purpose of this task is to understand what functional coverage means by performing a simple coverage for the reset signal (`tb_reset_n` in the testbench). As mentioned previously, the reset in this system is active low, therefore a very simple coverage is to find out whether the reset signal assumes both active and deactive states (0 and 1 respectively).

Here's how you will go about it. Open the file `test_top.sv` and uncomment the 3 lines below the comment `// Task 1`, and save the file. This will enable the **coverpoint** `reset` in the testbench. Now you can click on the `Compile Out of Date` button shown in Figure 20 to just compile the file you modified.



Figure 20: Compile Out of Date button

If you kept the simulation running in the previous stage as suggested above, you can simply restart the simulation by clicking on the `Restart` button in the toolbar (Figure 21), and then on the `Run -All` button.



Figure 21: Restart button

You can see now that the report shows that the functional coverage has failed with 50% coverage. Why is that? You can find out by opening the `Covergroups` tab in the project by following `View->Coverage->Covergroups`. You only need to do this once, since this tab will remain open and will be updated as you do your changes. Figure 22 demonstrates how things should look like on your end.

Note: If you are not able to expand the tree in the `Covergroups` tab, then right click on the module `serial_to_parallel_test` and select `Report...`. This will open the `Functional coverage report` window which you can set up according to Figure 23, and then click `OK`. Go to the end of the report and there you can see the coverage status for each coverpoint (the lines starting with the word "bin").

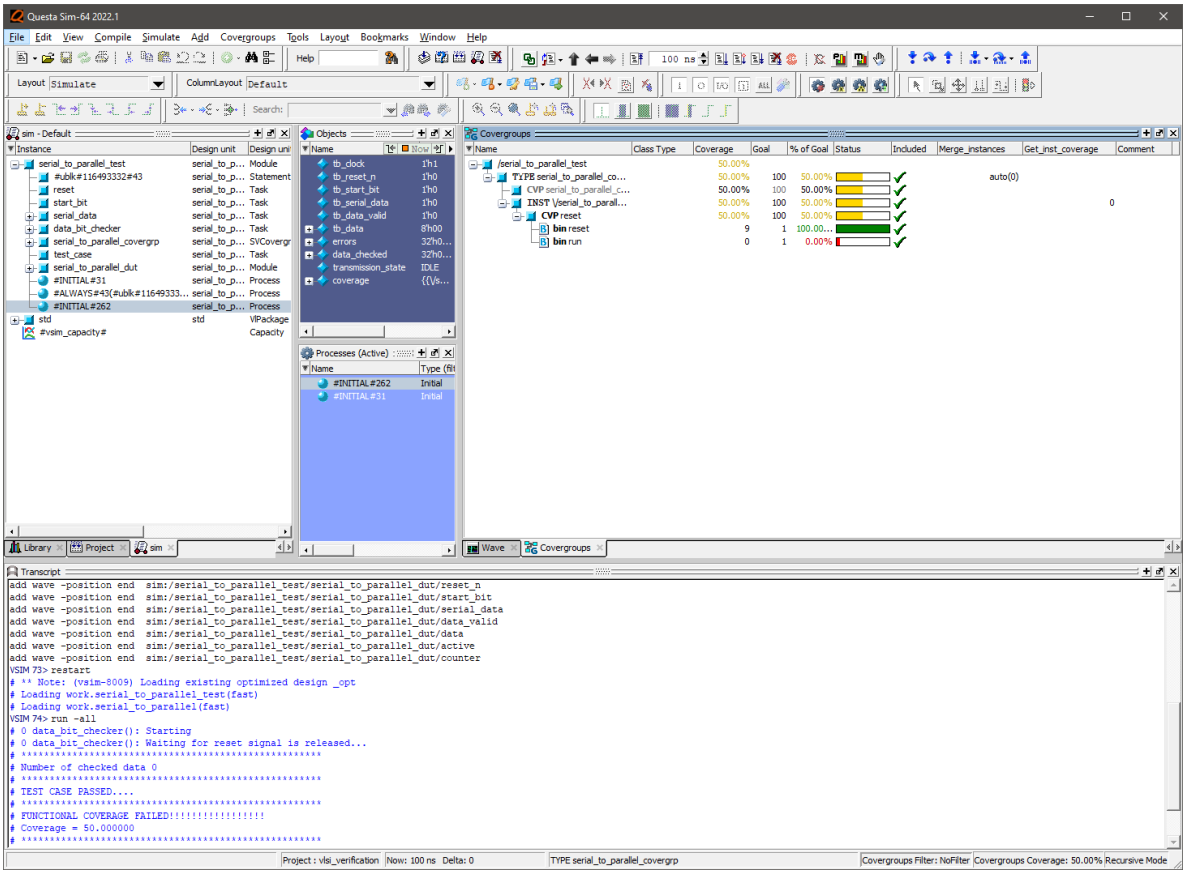


Figure 22: Covergroups

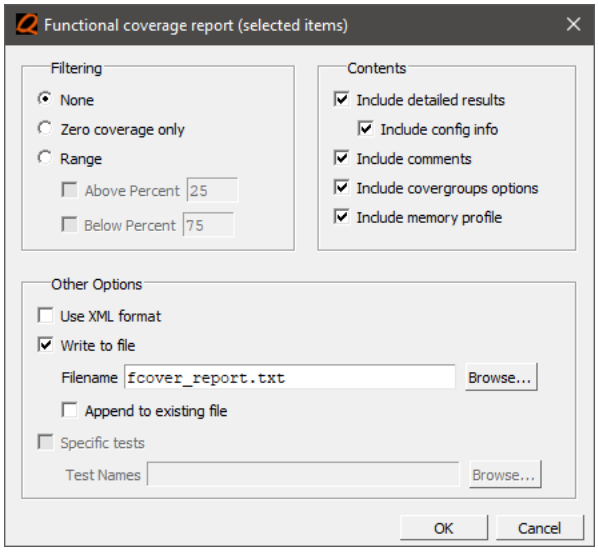


Figure 23: Coverage report

What does this mean? Try to make sense of the interplay between the lines you uncommented and the results of the `Covergroups` tab. `Hint`: Check out the reset signal's activity.

Next, switch back to the `test_top.sv` file and uncomment the line bellow the comment `// Task 2`. This will cause the reset signal to be deasserted (i.e. get deactivated by going high) after 2 clock cycles.

This means the system is reset for 2 cycles and then can function normally. Now compile, restart and run simulation again. Check the `Wave` tab and convince yourself that this was actually the case by inspecting the `reset_n` signal.

Did the functional coverage pass or not? If yes, what changed to cause this and what do you think the coverpoint in the lines below `// Task 1` that you uncommented is for?

6.3 A more complicated coverage test

In this task you will attempt to perform a more complex coverage test involving the start signal (`tb_start_bit`) and the simple state machine that was introduced in `// Section 4`. Uncomment the 5 lines below the `// Task 3` comment and rerun the simulation again. What effect did this have on coverage? What does the `coverpoint` you just uncommented try to cover?

Create a 1-cycle pulse on the start signal with a delay of 0 by putting the code after the `// Task 4` comment. Check the waveform and confirm your changes were correct. Does this affect the coverage in any way? Why?

6.4 Cross coverage

In the previous tasks you were only running coverage on individual signals. But what if you want to expand the coverage test to check whether certain combination of signal values happen or not concurrently. For example, you might want to check to see if in your tests you have a scenario in which a reset happens while the design is already working on a current conversion. In this way you make sure that your system works not only when it is reset at the beginning, but also when it is reset while it is working.

In order to do something like what was described above we need to perform cross coverage. Uncomment the line immediately following `// Task 5` and rerun the simulation. This will add cross coverage between the reset and the state machine. Inspect the coverage now (Figure 24). Why isn't it at 100%? Can you understand the 6 bins that were automatically created for you? What does each one cover?

Check out the new bins that have 0% coverage in the `Covergroups` tab. Can you create more changes on the start and reset signals (put below `// Task 6` comments) in a way that the coverage reaches 100%? Make sure you are able to explain how you managed to attain this result.

Note: After you're done, check the test case status printouts. Did you notice that by performing the cross coverage you uncovered a bug in the design? The next section is devoted to this issue.

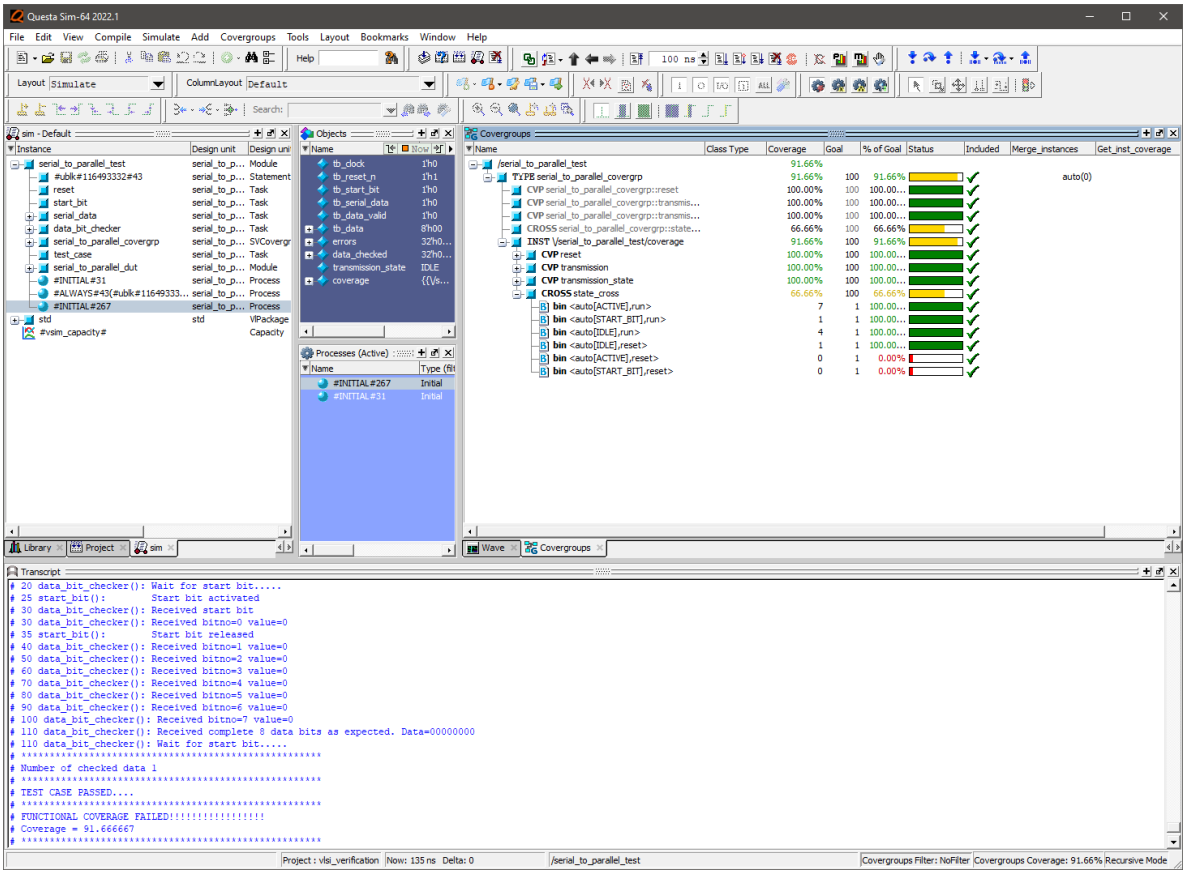


Figure 24: Cross coverage

6.5 Design bug

"TEST FAILED!!!!!!!!!!!!!!!!". That's a bummer isn't it? But cheer up. We will not let a small roadblock like this hamper us. Debugging to the rescue! The problem is that there is a very simple bug in the design that causes the test not to pass. It was fortunate that our coverage analysis actually *uncovered* this bug, and that's the whole point, isn't it?

You can find out what the error is by scrolling through the `Transcript` pane and searching for `ERROR`. When you find it you can double click on it or check the cycle stamp next to it. Check the waveform at that cycle and also the error message in the `Transcript` pane. Can you decrypt the message and fix the bug in the design? Why did the bug happen in the first place?

6.6 Coverage for the data signal

Now you will move on to the coverage for the data signal. The point is that you want to make sure the system works for different data bundles and you will do this by checking the coverage.

Try to send the data value 0xA2 after some delay (`// Task 7`), and make sure that it is accompanied by a proper start signal (with no interrupting reset). Further, uncomment the 4 lines below `// Task 8`. It should be fairly obvious what these lines try to cover. If not then ask around. Do you achieve full coverage?

Add more start bit and data stimuli after lines designated with `// Task 9` to attain a 100% coverage.

Finally, complete the coverage for data values by adding more bins for `// Task 8` so that other cases are also covered. Now try to get a 100% coverage by sending more data.

6.7 UVM testbench

You have already seen what a traditional verification scenario might look like in the previous sections. Here we will take a look at a more modern style of verification with UVM, which is more suited for the complex systems that you will come across in practice.

Stop the simulation by `Simulate->End Simulation`, switch to the `Library` tab, right click on `tb_top` and select `Simulate`. This will put the UVM testbench through simulation. Click on the module `serial_to_parallel_dut` and add its signals to the wave window, and run the simulation.

As you can see this is similar to the traditional testbench we just covered. Make sure to go through the files and try to understand how the testbench is organized. Even though our traditional testbench did a rather satisfactory job in this example, for more complicated designs this sort of testing mechanism gets unwieldy pretty quickly. Consequently, the UVM methodology is used in practice in real life scenarios and familiarity with its concepts is quite beneficial. So don't be lazy. To UVM and beyond!